System Verification and Validation Plan for Chest Scan

Team 16, Ace
Harrison Chiu
Hamza Issa
Ahmad Hamadi
Jared Paul
Gurnoor Bal

April 2, 2025

Revision History

Date	Author	Notes
4 November 2024	Harrison	Section 4
4 November 2024	Hamza	Section 5
4 November 2024	Gurnoor	Section 1-3
4 November 2024	Jared	Section 3, 5
4 November 2024	Ahmad	Section 5

Contents

1 Symbols, Abbreviations, and Acronyms **v**

2 General Information **1**
2.1 Summary . . . 1
2.2 Objectives . . . 1
2.2.1 Out of Scope for the V&V Plan . . . 2
2.3 Challenge Level and Extras . . . 2
2.3.1 Extras . . . 2
2.4 Relevant Documentation . . . 3

3 Plan **3**
3.1 Verification and Validation Team . . . 3
3.2 SRS Verification Plan . . . 4
3.2.1 Objectives . . . 4
3.2.2 Verification Checklist . . . 5
3.2.3 Methods . . . 6
3.2.4 Verification Criteria . . . 6
3.3 Design Verification Plan . . . 6
3.3.1 Objectives . . . 7
3.3.2 Verification Checklist . . . 7
3.3.3 Methods . . . 8
3.3.4 Verification Criteria . . . 8
3.4 Verification and Validation Plan Verification Plan . . . 9
3.4.1 Objectives . . . 9
3.4.2 Verification Checklist . . . 9
3.4.3 Methods . . . 10
3.4.4 Verification Criteria . . . 10
3.5 Implementation Verification Plan . . . 11
3.5.1 Objectives . . . 11
3.5.2 Verification Methods . . . 11
3.5.3 Verification Criteria . . . 12
3.6 Automated Testing and Verification Tools . . . 12
3.6.1 Objectives . . . 12
3.6.2 Tools and Methods . . . 13
3.6.3 Success Criteria . . . 13
3.7 Software Validation Plan . . . 14
3.7.1 Objectives . . . 14
3.7.2 Validation Methods . . . 14
3.7.3 Validation Criteria . . . 15

4 System Tests **15**
4.1 Tests for Functional Requirements . . . 15
4.1.1 Handling Input Data . . . 15

4.1.2 Disease Identification and Classification . . . 16
4.1.3 Report Generation . . . 18
4.1.4 Display of Heatmaps and Reports . . . 19
4.1.5 User Interface Access . . . 19
4.1.6 Data Storage and Security . . . 20
4.1.7 Authentication and Authorization . . . 21
4.2 Tests for Nonfunctional Requirements . . . 22
4.2.1 Appearance Requirements . . . 22
4.2.2 Usability and Humanity Requirements . . . 23
4.2.3 Usability and Humanity Requirements . . . 25
4.2.4 Performance Requirements . . . 25
4.2.5 Security Requirements . . . 27
4.2.6 Supportability Requirements . . . 28
4.2.7 Cultural Requirements . . . 29
4.3 Untested Non-Functional Requirements . . . 29
4.4 Traceability Between Test Cases and Requirements . . . 30
4.5 Traceability Between Test Cases and Requirements . . . 31

5 Unit Test Description **32**
5.1 Unit Testing Scope . . . 32
5.2 Tests for Functional Requirements . . . 34
5.2.1 Data Preprocessing Module . . . 34
5.2.2 ChestXrayDataset Module . . . 35
5.2.3 Model Architecture Module . . . 36
5.2.4 Training and Inference Module . . . 37
5.2.5 Model Persistence Module . . . 38
5.2.6 Data Download and Extraction Module . . . 39
5.3 Tests for Nonfunctional Requirements . . . 40
5.3.1 Performance Testing - PredictiveModel Module . . . 41
5.3.2 Interpretability Testing - DiagnosticAnalysis Module . . . 41
5.3.3 Fairness and Bias Testing - PredictiveModel Module . . . 41
5.3.4 Security Testing - Training and Inference Module . . . 41
5.4 Traceability Between Test Cases and Modules . . . 42

6 Appendix **43**
6.1 Symbolic Parameters . . . 43
6.2 Usability Survey Questions? . . . 44
6.2.1 User Experience Survey . . . 44
6.2.2 Additional Feedback: . . . 44
6.3 Question 1 . . . 45
6.4 Question 2 . . . 45
6.5 Question 3 . . . 46
6.6 Question 4 . . . 47

List of Tables

2 Traceability Matrix - Functional Requirements to Test Cases 30
3 Traceability Between Non-Functional Requirements and Test Cases 31
4 Legend for Module Labels 42
5 Traceability Matrix for Functional Requirement Test Cases (FRTC) 43
6 Traceability Matrix for Non-Functional Requirement Test Cases (NFRTC) . 43

1 Symbols, Abbreviations, and Acronyms

Table 1, includes the definitions and descriptions of all relevant symbols, abbreviations and acronyms used in this VnV Plan document.

Symbol, Abbreviation or Acronym	Definiton or Description
ML	Machine Learning: A branch of artificial intelligence that involves the use of algorithms to allow computers to learn from and make predictions based on data. This is a core technology used in the project for analyzing chest X-rays.
DL	Deep Learning: A subset of machine learning involving neural networks with many layers, used to analyze various types of data, including images.
DICOM	Digital Imaging and Communications in Medicine: A standard for transmitting, storing, and sharing medical imaging information. It is used to manage medical images in the proposed solution.
CNN	Convolutional Neural Network: A type of deep learning model specifically designed for processing structured grid data like images, used in the project for chest X-ray analysis.
EHR	Electronic Health Record: A digital version of a patient's paper chart, used for storing patient information and history that may be integrated with the proposed solution.
API	Application Programming Interface: A set of rules and protocols for building and interacting with software applications, enabling the integration of the proposed solution with other systems.
MC	Mandated Constraints: Various constraints placed on the project's proposed solution that must be adhered to throughout the development process.
FR	Functional Requirement: A requirement that specifies what functionality the project's proposed solution must provide to meet user needs.
NFR	Nonfunctional Requirement: A requirement that specifies criteria that can be used to judge the operation of a system, rather than specific behaviors (e.g., performance, usability).
BUC	Business Use Case: A scenario that describes how the proposed solution can be used within a business context to achieve specific goals.
PUC	Product Use Case: A scenario that details how an individual user will interact with the proposed solution to achieve specific tasks.
MVP	Minimum Viable Product: A version of the proposed solution that includes only the essential features required to meet the core needs of the users and stakeholders.

MG	Module Guide
MIS	Module Interface Specification
PoC	Proof of Concept
SRS	Software Requirements Specification
FRTC	Functional Requirements Test Case
NFRTC	Nonfunctional Requirements Test Case
VnV	Verification and Validation

This document outlines the verification and validation (VnV) plan for the project's proposed solution. It begins with a summary of the software's general functions, the objectives of the VnV plan (including in-scope and out-of-scope elements), and references relevant documentation.

It lists the VnV team members and describes the process for verifying the SRS, design, VnV plan, and implementation, including the automated testing tools used. The VnV roadmap starts with verifying the SRS, followed by the design, VnV plan, and implementation. After these steps, the software will undergo validation, with milestones achieved through automated tools and guidance from the project supervisor. It also shows the tests done (system and unit tests) to each major module to verify its functionality and output.

2 General Information

2.1 Summary

The software being developed is the **Chest Scan** AI diagnostic tool, designed to assist radiologists by automating the classification of chest X-ray images and generating structured diagnostic reports. The system leverages convolutional neural networks (CNNs) to detect and classify multiple thoracic diseases—including pneumonia, cardiomegaly, and effusions—within a single scan. This multi-class classification capability enhances diagnostic efficiency and helps radiologists manage large volumes of medical imaging. The tool will include user authentication, heatmap visualization (via Grad-CAM), confidence scoring, and PDF reporting through a web interface.

2.2 Objectives

The V&V Plan for the Chest Scan system targets three major goals: diagnostic reliability across disease categories, secure handling of sensitive clinical data, and intuitive system usability. Each objective is measurable and traceable to the system's core requirements. They guide testing for both functional correctness and compliance with healthcare regulations.

1. **AI Model Accuracy and Diagnostic Reliability**
    - Verify that the CNN model can classify chest X-rays across multiple disease labels (e.g., pneumonia, cardiomegaly, edema) with high precision and recall per class.
    - Confirm that class-specific false positive and false negative rates remain below thresholds defined in the SRS (e.g., $< 10\%$ FP for cardiomegaly).
    - Ensure that heatmaps and confidence scores correspond to the regions of the X-ray that support each prediction, enabling radiologist verification.
2. **Data Security and Access Control**
    - Confirm that all personally identifiable health data is encrypted both at rest and during transmission, using standards such as AES-256 and HTTPS.

- Verify that only authorized users (e.g., clinicians, researchers) can access system outputs via token-based authentication.
- Validate that the system is aligned with HIPAA and PIPEDA standards by conducting compliance reviews and verifying that audit logs, encryption, and access control procedures are implemented correctly.

3. **Usability and System Accessibility**
   - Verify that radiologists can intuitively upload images, receive multi-label predictions, view corresponding heatmaps, and download PDF reports.
   - Confirm that the UI supports readability, keyboard navigation, and mobile responsiveness for accessibility.
   - Validate seamless integration with imaging systems (e.g., PACS) for DICOM file upload and metadata retrieval.

2.2.1 Out of Scope for the V&V Plan

While essential to future development, the following areas are currently beyond the scope of this V&V plan:

- **Industry-Standard UI/UX Optimization:** The plan does not include formal usability heuristics testing or A/B interface comparisons.
- **Third-Party Library Validation:** The plan assumes correctness of well-established open-source ML libraries (e.g., PyTorch, TensorFlow).
- **Full Load and Stress Testing:** The MVP is scoped for single-user workflows; multi-user scalability testing will be deferred.

2.3 Challenge Level and Extras

This project is considered high challenge due to its simultaneous emphasis on medical correctness, model generalizability, regulatory compliance, and usability.

AI Complexity: Building a CNN capable of handling multi-label classification on chest X-rays requires careful dataset curation, bias control, and threshold tuning.

Healthcare Compliance: Ensuring secure data handling practices compliant with HIPAA and PIPEDA adds architectural and legal overhead.

Clinical Relevance: Presenting diagnostic results in a format interpretable by radiologists, including justification visualizations (e.g., heatmaps), imposes a high bar for usability and interpretability.

2.3.1 Extras

If project capacity permits, the team will pursue the following enhancements:

- **Usability Testing with Radiologists:** Targeted sessions will be held to assess interface clarity and diagnostic report interpretation speed.
- **Research Paper and Documentation:** Full technical documentation and a research summary will be published detailing model design, architecture, training pipeline, and V&V outcomes.

2.4 Relevant Documentation

The following deliverables provide traceable, testable definitions for both requirements and architecture:

- **Software Requirements Specification (SRS):** Defines functional (e.g., FR1-FR6) and nonfunctional (e.g., SR1-SR3, AR1-AR3) requirements with fit criteria and rationale.
- **Problem Statement Document:** Articulates the motivation, target stakeholders, and scope of the CNN chest X-ray diagnostic system.
- **Module Guide (MG):** Describes the modular breakdown of the system, including the AI engine, API controller, frontend renderer, and report generator.
- **Module Interface Specification (MIS):** Specifies interfaces, state transitions, and exported functions of each module. Used to define V&V unit test boundaries.

3 Plan

In this section, we will lay out the roadmap for ensuring each part of the project meets the intended standards through verification and validation. Responsibilities are assigned to team members for each task within the V&V process. This plan will address how we will confirm that the project's requirements, design documents, V&V plan, and final implementation align with the specified criteria. Tools and automated testing solutions that support our verification efforts will also be outlined. Additionally, a detailed process for validating the software's functionality and performance will be included. Our verification tasks will be presented in a structured sequence, moving from requirements analysis to design inspection, implementation testing, and review of the V&V process itself. Requirements verification will continue throughout development, with design checks occurring as the code is produced. Validation of the implementation will be an ongoing effort, eventually leading to a proof-of-concept phase and final project demonstrations.

3.1 Verification and Validation Team

VnV Team Member Name	Summary of Role(s)
Amirhossein Sabour	Advisor; oversees V&V strategy, ensures documentation quality, and provides final validation input. Reviews model performance metrics and compliance with clinical AI validation standards.
Other Design Teams	Peer reviewers. Raise model validation issues (e.g., poor class-wise recall), documentation inconsistencies, or usability oversights through structured feedback and bug reports.
Gurnoor Bal	Leads verification of CNN model outputs, including confusion matrix analysis, precision/recall per disease class, and threshold sensitivity testing. Also supports validation of image upload and result display pipelines.
Jared Paul	Manages dataset quality verification tasks. Reviews for bias, annotation mismatches, and underrepresented disease categories. Conducts fairness and generalization checks.
Ahmad Hamadi	Responsible for backend logic and API validation. Verifies data flow integrity, image pre-processing, and robustness of prediction-serving endpoints. Also participates in Grad-CAM heatmap correctness validation.
Hamza Issa	Performs V&V of the web interface, focusing on frontend validation, usability compliance, and PDF generation. Participates in cross-checking classification results against reports.
Harrison Chiu	Oversees integration testing and multi-label output validation. Coordinates end-to-end test execution for model prediction, heatmap, report pipeline and leads usability testing sessions with stakeholders.

3.2 SRS Verification Plan

The SRS Verification Plan for the "Chest Scan" diagnostic tool ensures that all specified requirements are complete, accurate, and verifiable, particularly in the context of multi-label disease classification. This section outlines the approach to confirm that the SRS document meets quality standards, providing a solid foundation for subsequent design, implementation, and testing phases.

3.2.1 Objectives

The objectives of this SRS Verification Plan are to:

1. **Ensure Requirement Completeness:** Confirm that all essential functionalities, such as multi-class image classification, model confidence estimation, Grad-CAM visualizations, and diagnostic report generation, are fully and clearly specified in the SRS.

2. **Validate Requirement Clarity:** Verify that each requirement, including classi-

fication performance metrics (e.g., precision, recall, F1-score), is unambiguous and testable.

3. **Check Requirement Feasibility:** Assess that all technical features-especially those related to deep learning models-are realistically achievable given dataset availability and system constraints.
4. **Verify Traceability:** Ensure that each requirement can be traced through to design modules, test cases, and validation outputs.

3.2.2 Verification Checklist

To comprehensively verify the SRS, the following key elements will be checked:

1. **Functional Requirements**
   - Disease detection and classification across multiple diagnostic categories (e.g., pneumonia, pleural effusion, cardiomegaly, etc.).
   - Accuracy thresholds for class-wise detection defined using symbolic constants (e.g., `MIN_ACCURACY_THRESHOLD`).
   - Confidence score computation and annotation with classification results.
   - Integration of interpretability tools such as Grad-CAM heatmaps.
   - Diagnostic report generation with location-based annotations and condition summaries.
2. **Nonfunctional Requirements**
   - **Performance:** Symbolic benchmarks for classification latency, throughput, and expected accuracy ranges under load.
   - **Security:** HIPAA-compliant handling of sensitive data including encryption-at-rest, secure data transfer, and access control policies.
   - **Usability:** UI clarity, response times for clinical review, and interpretability of classification results.
   - **Reliability:** Uptime requirements, fallback behaviors in case of model failure or data corruption.
3. **Constraints**
   - Use of PyTorch and Grad-CAM as mandatory frameworks and tooling.
   - Compliance with public dataset licenses (e.g., CheXpert, MIMIC-CXR, NIH) and hospital system compatibility (PACS/HIS).
4. **Traceability and Identification**
   - All requirements assigned unique identifiers (e.g., `FR1`, `NFR2`) mapped to modules and future test cases.

- Inclusion of backward and forward traceability from requirements to test outputs and system modules.

5. **Glossary and Terminology**
   - Standardization of clinical terms (e.g., infiltrates, nodules), technical terms (e.g., Grad-CAM, ROC-AUC), and abbreviations.
6. **Assumptions and Dependencies**
   - Availability of annotated chest X-ray datasets (with multi-class labels) assumed.
   - Dependence on high-performance hardware (e.g., GPU-enabled environments) for real-time inference.

3.2.3 Methods

The following methods will be used to verify the SRS:

- **Requirement Review Meetings:** Cross-functional team discussions with domain experts to confirm all requirements support multi-class clinical use cases.
- **Traceability Matrix Construction:** All SRS items mapped to functional and non-functional test cases for validation and unit testing.
- **Cross-Referencing with MG and MIS:** Ensures decomposition of requirements into software modules and well-defined interfaces for each responsibility.

3.2.4 Verification Criteria

The SRS will be considered verified if:

- Requirements meet completeness, consistency, and clarity standards suitable for medical AI tools.
- Each requirement is covered in the test case traceability matrix with associated validation criteria (e.g., precision, recall, Grad-CAM heatmap visibility).
- All feedback from domain reviewers and stakeholders (e.g., radiologists, AI engineers) is addressed and resolved.

By following this updated SRS Verification Plan, the team will ensure that the diagnostic system's architecture supports reliable, explainable, and secure multi-disease classification suitable for healthcare deployment.

3.3 Design Verification Plan

This Design Verification Plan focuses on verifying that the system's modular structure and interface specifications, as outlined in the Module Guide (MG) and Module Interface Specification (MIS), align with the requirements in the SRS. Each module's functionality, integra-

tion, and adaptability to potential changes will be reviewed to ensure the system is robust, maintainable, and ready for clinical use.

3.3.1 Objectives

The objectives of this Design Verification Plan are to:

1. **Verify Modular Consistency with Requirements:** Ensure that each module in the MG fulfills its corresponding SRS requirements, including image processing, multi-class classification, Grad-CAM visualization, and secure access.
2. **Validate Information Hiding and Encapsulation:** Confirm that modules encapsulate implementation details (e.g., CNN model parameters, data loaders) to support abstraction and maintainability.
3. **Check Interface Compatibility:** Verify that module interfaces in the MIS support accurate data exchange between components like the frontend UI, backend API, and CNN classification engine.
4. **Assess Anticipated Changes and Traceability:** Ensure that anticipated design changes (e.g., swapping the model architecture or modifying output schema) are localized and traceable.

3.3.2 Verification Checklist

The following checklist covers key design aspects to be verified based on the MG and MIS documents:

1. **Module Decomposition**
   - Modules follow the information-hiding principle and align with layered design (e.g., ML layer, data layer, UI layer).
   - CNN architecture module includes breakdown of input layers, convolutional stacks, activations (e.g., ReLU), pooling layers, and softmax/multi-label classifiers.
   - Traceability matrix maps SRS classification requirements (e.g., FR1-FR4) to CNN Model and Grad-CAM modules.
2. **Module Interfaces (MIS)**
   - CNN module input interface accepts preprocessed X-ray tensors with consistent shape (e.g., `B x 1 x 224 x 224`).
   - Output interface returns a multi-class probability vector, interpretable labels, and Grad-CAM heatmaps.
   - Modules export constants like `CLASS_LABELS`, `CONFIDENCE_THRESHOLD`, ensuring no hard-coded literals.
3. **Anticipated and Unlikely Changes**

- Anticipated: Change in CNN architecture (e.g., swapping DenseNet with ResNet) should be confined to the ML module.
- Unlikely: Change to data modality (e.g., CT scans instead of X-rays) would affect preprocessing and UI assumptions-documented as unlikely to reduce over-design.

4. **Traceability and Use Hierarchy**
   - Each design element traced to one or more SRS requirements using the traceability matrix.
   - UI modules depend on Backend API, which depends on ML model, which in turn depends on DataLoader and Preprocessing modules (clean downward hierarchy).
5. **User Interface Design (if applicable)**
   - Image upload UI and Grad-CAM overlay visualizations are verified for compatibility with backend output format.
   - Diagnostic report UI tested for multi-class display layout (i.e., stacked or tabular outputs for multiple disease labels).

3.3.3 Methods

The following methods will be used to verify the design:

- **Design Walkthroughs:** Systematic reviews with the team to inspect modular boundaries, data flow, and CNN structure alignment with requirements.
- **Consistency Checks with SRS:** Ensure module responsibilities match SRS requirements, including disease classification, heatmap generation, and frontend reporting.
- **Interface Testing:** Simulate API calls or UI interactions using mock data to verify each module interface behaves as expected.
- **Traceability Matrix Review:** Confirm each FR/NFR is mapped to design modules and test plans for coverage and traceability.

3.3.4 Verification Criteria

The design is considered verified if:

- The CNN model structure, activation pipeline, and outputs align with SRS expectations and clinical interpretability.
- Modules demonstrate proper encapsulation of model logic, UI routines, and data transformations.
- Interface definitions are consistent, conflict-free, and aligned with MIS documentation.
- All traceability links between requirements and modules are complete and validated.

This plan ensures the system's design is ready for implementation, supporting flexibility, traceability, and correctness for real-world clinical deployment.

3.4 Verification and Validation Plan Verification Plan

This Verification and Validation (V&V) Plan Verification Plan outlines the process to ensure that the V&V document itself is thorough, accurate, and effectively designed to support all project goals. By verifying this V&V document, we aim to confirm that all sections—objectives, scope, methods, and criteria—are clearly defined and cover the requirements, design, and implementation needs outlined in the SRS, MG, and MIS documents.

3.4.1 Objectives

The objectives of this V&V Plan Verification Plan are to:

1. **Ensure Completeness of Verification and Validation Processes:** Verify that this V&V Plan includes all necessary sections to thoroughly validate the tool's requirements, design, and implementation.
2. **Confirm Clarity and Traceability:** Ensure that each section is clearly written, unambiguous, and easy to follow for all project stakeholders, with traceability to project documents.
3. **Validate Feasibility of Methods:** Check that the methods and criteria specified in the V&V Plan are feasible given project resources and timelines.
4. **Identify Gaps and Overlaps:** Ensure that all elements of the system are addressed without unnecessary repetition, minimizing gaps or redundant validations.

3.4.2 Verification Checklist

The following checklist identifies key elements in the V&V Plan to verify for completeness and accuracy:

1. **Objectives and Scope**
   - Verify that the V&V Plan includes well-defined objectives and scope covering all essential elements (SRS, MG, MIS).
   - Confirm that the plan covers functional and nonfunctional requirements, with a clear boundary on what is out of scope.
2. **Verification and Validation Methods**
   - Ensure that each section (e.g., SRS Verification Plan, Design Verification Plan) includes clearly defined methods and criteria for verifying requirements, design, and implementation.
   - Confirm that the V&V methods align with the constraints and objectives laid out in the project's SRS.

3. **Traceability and Consistency**
   - Check that the V&V Plan is traceable to all primary project documents (SRS, MG, MIS) with appropriate references, ensuring alignment and consistency across documents.
   - Ensure that each V&V Plan section is traceable back to specific requirements or design aspects from the SRS and MG, ideally via traceability matrices.
4. **Feasibility and Practicality of Methods**
   - Confirm that each verification and validation method in the V&V Plan is achievable within the project's resources, timeline, and technical constraints.
   - Check that tools, data, and stakeholder inputs needed for each V&V process are specified and accessible.
5. **Criteria for Success**
   - Ensure that the V&V Plan defines clear criteria for determining the success of each verification and validation step.
   - Verify that these criteria are realistic and aligned with project requirements, design goals, and intended system functionality.

3.4.3 Methods

To verify the V&V Plan:

- **Document Review Meetings:** Conduct meetings with project stakeholders and team members to review each section of the V&V Plan for clarity, feasibility, and traceability.
- **Cross-Referencing with Project Documents:** Check each verification and validation method against the SRS, MG, and MIS to confirm alignment and completeness.
- **Feasibility Assessment:** Evaluate the methods and tools proposed in the V&V Plan to ensure they are practical and achievable with available resources.

3.4.4 Verification Criteria

The V&V Plan will be considered verified if:

- All sections are complete, clearly written, and aligned with project documents (SRS, MG, MIS).
- The verification and validation methods are feasible and cover all essential elements.
- Stakeholders confirm that the V&V Plan is practical and provides a robust approach for testing and validating the tool.

This Verification Plan ensures that the V&V Plan itself is reliable, thorough, and ready to support a comprehensive validation process for the "Chest Scan" diagnostic tool.

3.5 Implementation Verification Plan

This Implementation Verification Plan outlines the steps to confirm that the "Chest Scan" diagnostic tool's final implementation accurately reflects its design, fulfills all specified requirements, and operates as intended in realistic clinical environments. The plan emphasizes evaluating model correctness, user experience, dataset adaptability, and multi-label classification robustness. It also verifies that the system supports interpretability and bias-reduction techniques across various disease classes.

3.5.1 Objectives

The objectives of the Implementation Verification Plan are to:

- **Verify Functional Accuracy:** Ensure that each implemented feature—including disease detection, diagnostic report generation, and heatmap generation—performs according to its design and aligns with the SRS functional requirements.
- **Confirm Model Stability and Reliability:** Assess the CNN's stability across training sessions using different dataset splits, ensuring that performance (e.g., accuracy, precision, recall) consistently meets symbolic thresholds (e.g., `ACC_THRESHOLD`).
- **Validate Dataset Adaptability and Generalization:** Confirm that the model performs well on real-world datasets (e.g., CheXpert, MIMIC-CXR) beyond its training data, ensuring broad applicability.
- **Ensure Security and Usability Compliance:** Verify encryption protocols, access control logic, and that the user interface is intuitive and accessible for clinical users.
- **Evaluate Bias Mitigation:** Verify that training pipelines implement data balancing, augmentation, or stratification techniques to reduce biases from class imbalance or dataset skew.

3.5.2 Verification Methods

The following methods will be employed to verify implementation:

- **Unit Testing:** Each backend module, including image preprocessing, classification, report formatting, and UI display components, will be tested using automated unit tests tied to unique SRS identifiers.
- **Integration Testing:** Functional connections between subsystems (e.g., frontend upload → backend classifier → report UI) will be validated. These tests confirm correct data transfer, error handling, and system communication flows.
- **System Testing:** The entire platform will be tested in a near-production environment. Evaluation will include:
  - Accuracy metrics (precision, recall, F1-score) on a holdout dataset.
  - Classification confusion matrix per disease class.

- Model explainability checks using Grad-CAM overlays.
- Load testing with multiple uploads to verify concurrency.
- HIPAA-aligned encryption and access audits.

3.5.3 Verification Criteria

The implementation will be considered verified if:

- **Pass Rate:** 100% of unit and integration tests pass without critical or high-severity errors.
- **Performance Thresholds:** Multi-class CNN meets symbolic metrics such as `ACC_THRESHOLD`, `PREC_THRESHOLD`, and `RECALL_THRESHOLD` for top-priority disease classes.
- **Model Stability:** Results from multiple runs with different seeds and data splits show consistent classification performance within acceptable variance.
- **Bias Detection and Mitigation:** Validation shows no statistically significant performance drop between underrepresented and overrepresented classes (p-value ¡ 0.05).
- **Clinical Usability and Security:** Stakeholders confirm that the UI is intuitive, reports are interpretable, and data access follows security standards (AR2, SR3).

By following this Implementation Verification Plan, the project team will ensure that the final deployed system behaves reliably across use cases, meets clinical needs, adapts to real-world data, and remains interpretable and secure throughout operation.

3.6 Automated Testing and Verification Tools

Automated testing will ensure consistent verification of the "Chest Scan" diagnostic tool's functionality, classification accuracy, security, and fairness throughout its development. Given the complexity of a multi-class CNN in a clinical setting, automated tools will support reproducibility, precision monitoring, and protection against model drift or biased outputs. Tools will be selected to optimize testing efficiency, track model behavior, and enforce project constraints through a continuous integration pipeline.

3.6.1 Objectives

The goals of automated testing are to:

- **Enhance Testing Efficiency:** Use automated testing suites to reduce manual verification time and catch regressions early in the development process.
- **Support Continuous Integration and Deployment (CI/CD):** Automate testing workflows to verify code and model behavior on each update.
- **Track Model Performance and Fairness:** Implement evaluation scripts that monitor classification metrics (e.g., precision, recall), dataset bias, and output disparities across disease labels.

- **Evaluate Adversarial Robustness:** Test model performance against perturbed or low-quality images to ensure reliability under real-world imaging variability.

3.6.2 Tools and Methods

The following tools and methods will be employed during the project:

- **Unit Testing:**
  - `pytest`, `unittest`: Validate core functionalities such as preprocessing routines, model inference, and data formatting logic.
  - Test metrics include symbolic constants like `ACC_THRESHOLD` and `RECALL_THRESHOLD` defined in alignment with SR1.
- **Model Evaluation Scripts:**
  - Custom Python scripts to automatically compute:
    * Multi-class confusion matrix and per-class precision/recall/F1-scores.
    * Grad-CAM heatmap visual inspection and feature localization checks.
    * Bias detection using demographic parity or subgroup accuracy tests (for future scalability).
  - Tools like `scikit-learn`, `captum`, and `Fairlearn` will be used for metric calculation and fairness evaluation.
- **UI and Integration Testing:**
  - `Selenium`: Automated interaction testing for the frontend, validating image uploads, report visibility, and user flow.
  - `React Testing Library`: Unit tests for frontend components, such as result modals and image previews.
- **Continuous Integration Pipeline:**
  - `GitHub Actions`: Executes linting, test suites, and evaluation scripts on each pull request.
  - `Flake8`, `Black`, `ESLint`: Enforce Python/JS code style and formatting rules.
  - Auto-reporting system logs test results and accuracy drops to alert developers of performance degradation.

3.6.3 Success Criteria

Automated testing will be considered effective if:

- The CI pipeline completes without failures across unit, integration, and evaluation stages.

- Model performance consistently exceeds defined symbolic thresholds (`ACC_THRESHOLD`, `F1_THRESHOLD`) during automated testing.
- Bias and robustness test scripts identify performance disparities early and allow corrective interventions (e.g., resampling or retraining).
- Functional UI tests ensure that no deployment breaks the image upload or report display pipeline.

Through this automation framework, the project will maintain high-quality, reproducible testing while improving classification accuracy, system robustness, and fairness in clinical AI diagnostics.

3.7 Software Validation Plan

This Software Validation Plan outlines the approach to confirm that the "Chest Scan" tool meets all functional and nonfunctional requirements, particularly in its role as a multi-class classification system for chest X-ray diagnostics. The plan incorporates quantitative and qualitative methods to ensure accuracy, fairness, usability, and clinical applicability.

External datasets (e.g., MIMIC-CXR, CheXpert) will be used to validate classification performance across multiple disease categories, while user testing with radiologists will provide real-world feedback on system usability and workflow integration.

3.7.1 Objectives

The objectives of the Software Validation Plan are to:

- **Validate Requirement Alignment:** Confirm that all functional requirements (FR1-FR4) and nonfunctional requirements (SR0, SR1, SR3) are fulfilled through structured task-based evaluations and stakeholder reviews.
- **Assess Multi-Class Classification Performance:** Use independent validation sets to assess precision, recall, F1-score, and confusion matrix per disease class, ensuring that model outputs meet diagnostic standards.
- **Gather Stakeholder Feedback:** Leverage Rev 0 and Rev 1 demonstration sessions with supervisors, clinicians, and developers to incorporate iterative feedback and refine the tool.
- **Confirm Clinical Usability:** Validate that the web interface and reporting pipeline align with radiologist workflows, supporting intuitive review of AI-generated reports and Grad-CAM visualizations.

3.7.2 Validation Methods

The following methods will be employed to confirm software effectiveness:

- **Task-Based Inspections:** Perform test cases simulating real-world scenarios such as uploading chest X-rays, viewing disease predictions, and interpreting Grad-CAM heatmaps. These tasks ensure diagnostic and interpretability goals are met.
- **Model Validation on External Datasets:** Run the trained CNN on held-out CheXpert and MIMIC samples and compute multi-class confusion matrices, per-class F1-scores, and overall accuracy to evaluate alignment with SR0 and SR1 thresholds.
- **Demo and Review Sessions:** Present working implementations in Rev 0 and Rev 1, gathering feedback from supervisors and potentially clinical advisors to assess requirement completeness.
- **User Testing with Radiologists:** Conduct observational usability studies, asking participants to navigate the tool, interpret results, and report on clarity, confidence, and workflow impact.

3.7.3 Validation Criteria

The software will be considered validated if:

- CNN model achieves symbolic accuracy and precision thresholds (e.g., `ACC_THRESHOLD` ≥ 0.90) across disease classes on independent datasets (SR0).
- Feedback from stakeholders (Rev 0/1) confirms compliance with both functional and nonfunctional expectations.
- Radiologist usability feedback indicates that the interface is interpretable, reliable, and clinically usable in simulated workflows.

This validation plan ensures the tool's real-world readiness through comprehensive evaluation of both performance metrics and user experience, aligning with the project's clinical goals and SRS-defined criteria.

4 System Tests

This subsection presents the system test cases categorized by areas of functionality. Each test validates a key feature described in the functional requirements. Tests span input handling, multi-class disease identification, report generation, visualization, user access, data protection, and authorization. References to SRS functional requirements are included in the derivations.

4.1 Tests for Functional Requirements

4.1.1 Handling Input Data

FRTC1
Title: Chest X-ray Image Input Acceptance
Control: Manual

Initial State: The system is initialized and ready to receive input through the web interface.
Input: A valid chest X-ray image in supported formats (DICOM, JPEG, or PNG).
Output: The image is successfully accepted, read, and queued for analysis with no errors or crashes.
Test Case Derivation: Derived from FR1 (The system shall accept and read chest X-ray images as input).
How the test will be performed:

1. Launch the system and ensure it's ready to receive inputs.
2. Use the upload interface to provide a valid chest X-ray image.
3. Confirm the image is accepted into the system for processing.
4. Monitor for any frontend or backend anomalies during the upload.

FRTC2
Title: Invalid Chest X-ray Image Format Rejection
Control: Manual
Initial State: The system is initialized and awaiting file upload.
Input: A file in an unsupported or corrupted format (e.g., .txt, .docx, or corrupted .jpg).
Output: The system rejects the file with a descriptive error message and logs the failure.
Test Case Derivation: Derived from FR1.
How the test will be performed:

1. Attempt to upload invalid image files.
2. Confirm that the UI blocks the upload and displays an appropriate error message.
3. Verify error handling is logged in the system backend.
4. Ensure no processing pipeline is triggered.

4.1.2 Disease Identification and Classification

FRTC3
Title: Correct Multi-Class Disease Classification
Control: Automatic (with manual verification)
Initial State: The multi-label classification model is deployed and active.
Input: Chest X-ray images with expert-annotated ground truth for multiple diseases.
Output: The model correctly identifies each disease label per image with per-class accuracy above 90%.
Test Case Derivation: Based on FR2.
How the test will be performed:

1. Load test dataset with known multi-disease labels (e.g., pneumonia + effusion).
2. Run the classification model and log predicted outputs.

3. For each disease class, compute accuracy = TP / (TP + FN + FP).
4. Check that every class's accuracy $\geq$ 90%.

FRTC4
Title: Disease Absence Correct Classification
Control: Automatic
Initial State: Model is deployed and ready.
Input: Chest X-ray images from healthy individuals with no labeled abnormalities.
Output: Model confirms absence of disease with accuracy above 90%.
Test Case Derivation: From FR2.
How the test will be performed:

1. Submit a set of verified normal chest X-ray images.
2. Record model outputs across all disease labels.
3. Confirm that no disease is incorrectly marked as present.
4. Compute false positive rate and verify it remains below tolerance.

FRTC5
Title: Ambiguous Image Classification (Multi-Label)
Control: Automatic (verified manually)
Initial State: Model and supporting UI loaded.
Input: Chest X-ray showing signs of multiple conditions (e.g., cardiomegaly + pleural effusion).
Output: Model correctly assigns multiple labels with balanced precision and recall across classes.
Test Case Derivation: Based on updated SRS scope (multi-label classification support).
How the test will be performed:

1. Use an ambiguous image with verified multiple diagnoses.
2. Run the classifier and extract all predicted labels.
3. Calculate precision, recall, and F1 for each label.
4. Ensure metrics meet performance thresholds $\geq$ 0.80.

FRTC6
Title: Robustness on Poor-Quality Images
Control: Automatic
Initial State: Model deployed.
Input: Noisy, low-contrast, or partially occluded chest X-ray image.
Output: Model attempts classification with degradation in accuracy within acceptable bounds (e.g., $\geq$ 70% per-class precision).

Test Case Derivation: From FR5 robustness requirement.
How the test will be performed:

1. Preprocess a test image with simulated noise or blur.
2. Submit image to system.
3. Record predictions and calculate performance degradation.
4. Confirm outputs remain within expected precision thresholds.

FRTC7
Title: Grad-CAM Heatmap Generation for Predictions
Control: Manual review
Initial State: A prediction with positive classification has been generated.
Input: Processed chest X-ray image with positive disease label (e.g., pneumonia).
Output: Grad-CAM heatmap is generated and highlights anatomical regions relevant to the classification.
Test Case Derivation: Interpretability requirement from FR3.
How the test will be performed:

1. Trigger heatmap generation post-inference.
2. Display Grad-CAM overlay to user.
3. Have medical reviewer confirm region aligns with pathology location.
4. Record alignment feedback to validate model transparency.

4.1.3 Report Generation

FRTC8
Title: Report Generation Error Handling
Control: Automatic
Initial State: Disease identification partially fails or returns incomplete results.
Input: A chest X-ray image where disease classification returns incomplete or no output.
Output: The system generates a report that clearly marks the failure, e.g., "Unable to determine finding for Region X." No system crash occurs.
Test Case Derivation: Derived from FR4 - Robust reporting under partial failure.
How the test will be performed:

1. Simulate failure in the disease identification model (e.g., forcibly bypass model return).
2. Attempt to generate the report.
3. Verify the report explicitly communicates which findings could not be produced.
4. Ensure formatting, layout, and system performance are unaffected.

4.1.4 Display of Heatmaps and Reports

FRTC9
Title: Heatmap Display on Chest X-ray Images
Control: Manual
Initial State: Heatmaps are generated successfully by Grad-CAM.
Input: A set of X-ray images with corresponding heatmaps.
Output: The UI overlays the correct heatmap aligned with the anatomical region corresponding to the diagnosis.
Test Case Derivation: Derived from FR3 - Visual explainability.
How the test will be performed:

1. Upload a verified test image with known labels.
2. View the generated heatmap overlay on the web interface.
3. Compare heatmap to radiologist-provided annotation.
4. Confirm localization and interpretability are accurate.

FRTC10
Title: Diagnostic Report and Heatmap Access via Web Interface
Control: Manual
Initial State: The system has generated and stored reports and heatmaps.
Input: Authenticated user request via the frontend dashboard.
Output: Interface displays both diagnostic report and heatmap in structured, accessible format.
Test Case Derivation: Derived from FR6 - User report accessibility.
How the test will be performed:

1. Log in to the application.
2. Navigate to patient case history.
3. Open a diagnostic report entry and inspect the associated heatmap.
4. Ensure both are visible, scrollable, and downloadable.

4.1.5 User Interface Access

FRTC11
Title: Authorized User Access to Diagnostic Results
Control: Manual
Initial State: The system is online, with at least one authorized user and patient report in the database.
Input: Valid login credentials (radiologist account).
Output: Full access granted to diagnostic results through the UI.

Test Case Derivation: Derived from FR7 - Role-based access to diagnostic data.
How the test will be performed:

1. Log in using a valid radiologist account.
2. Navigate to patient records.
3. Confirm ability to view diagnostic reports and overlays.
4. Confirm UI responsiveness and secure data retrieval.

FRTC12
Title: Unauthorized User Access Denied
Control: Manual
Initial State: The system has a record of authorized users.
Input: Invalid or unauthorized user login attempt.
Output: Access is denied with an informative error message.
Test Case Derivation: Derived from FR7 - Access restrictions for non-privileged users.
How the test will be performed:

1. Attempt login using a non-registered email.
2. Attempt login using a correct email but wrong password.
3. Confirm access is denied and appropriate errors are shown (e.g., "Invalid credentials").
4. Validate system logs failed access attempts.

4.1.6 Data Storage and Security

FRTC13
Title: Secure Storage of Patient Data and Diagnostic Results
Control: Manual and Automatic
Initial State: Database is populated with sample X-ray cases and diagnostic reports.
Input: Backend request to access, write, and update stored data.
Output: Data is retrieved/stored securely, with encryption and access logs verified.
Test Case Derivation: Derived from FR8 - Data security and patient confidentiality.
How the test will be performed:

1. Insert mock patient records into the database.
2. Run access queries through both backend services and simulated UI sessions.
3. Confirm proper handling and encryption (check DB layer and storage).
4. Verify access logs for each retrieval.

FRTC14
Title: Data Encryption in Storage and Transit

Control: Automatic
Initial State: System uses TLS/HTTPS and encrypted storage backend.
Input: User uploads and retrieves sensitive patient data.
Output: Data is encrypted end-to-end with no exposure of plaintext during transmission or at rest.
Test Case Derivation: Derived from FR9 - HIPAA-compliant encryption.
How the test will be performed:

1. Upload patient files via UI or backend.
2. Monitor network traffic using Wireshark - confirm HTTPS is used.
3. Query stored files and inspect encryption state on disk.
4. Validate that no part of the patient data exists unencrypted.

4.1.7 Authentication and Authorization

FRTC15
Title: User Authentication with Valid Credentials
Control: Manual
Initial State: System has at least one registered user.
Input: Valid username and password.
Output: User is authenticated and redirected to dashboard.
Test Case Derivation: Derived from FR7 - User authentication.
How the test will be performed:

1. Submit correct credentials on login form.
2. Confirm successful authentication.
3. Verify user lands on dashboard.
4. Check that a session token or cookie is generated.

FRTC16
Title: User Authentication with Invalid Credentials
Control: Manual
Initial State: System is live with authentication services running.
Input: Invalid login credentials.
Output: Authentication fails, and an error message is shown.
Test Case Derivation: Derived from FR7 - Authentication security.
How the test will be performed:

1. Enter wrong credentials repeatedly.
2. Confirm failure response (e.g., "Wrong username or password").

3. Observe system behavior on multiple attempts - check if lockout or CAPTCHA is triggered.

FRTC17
Title: Role-Based Access Control Enforcement
Control: Manual
Initial State: System has multiple users with defined roles (e.g., admin, radiologist, technician).
Input: Role-specific actions performed under different user accounts.
Output: System permits only actions within the role scope.
Test Case Derivation: Derived from FR7 - Role-based access enforcement.
How the test will be performed:

1. Log in as different user types.
2. Attempt actions outside their allowed scope (e.g., technician viewing admin panel).
3. Verify access is denied and system logs the attempt.
4. Confirm actions within role scope are permitted.

4.2 Tests for Nonfunctional Requirements

This subsection includes test cases that evaluate the system's appearance, usability, interpretability, and performance, as described in the SRS nonfunctional requirements. These tests are essential to ensuring the system is accessible, trustworthy, and usable in real-world settings.

4.2.1 Appearance Requirements

NFRTC1
Title: Verification of Calming Color Scheme
Control: Manual
Initial State: The web application is deployed and accessible from a browser.
Input: A user accesses the live system on a browser under default theme settings.
Output: The UI displays a cool and calming color palette, primarily using white and soft blue tones.
Test Case Derivation: Derived from NF-AR1 - The application shall use a calming color scheme to support clinician usability and reduce eye strain.
How the test will be performed:

1. Open the application in a modern browser under standard lighting.
2. Review the color palette used across login, dashboard, and report views.
3. Record the dominant background and highlight colors.

4. Conduct a brief evaluation with radiologists and designers, using a 5-point scale to rate visual comfort and clarity.
5. Pass criteria: At least 80% of users rate visual experience 4 or higher.

NFRTC2
Title: Consistency of Simple and Uncluttered Style
Control: Manual
Initial State: Web app is operational and fully loaded with all components.
Input: Navigation across multiple interface modules (login, dashboard, image viewer, report viewer).
Output: UI layout remains consistent and free of clutter; visual hierarchy is preserved.
Test Case Derivation: Derived from NF-SR1 and NF-SR2 - The software must present a clean interface with consistent layout and spacing across all pages.
How the test will be performed:

1. Access each core module of the UI as a user.
2. Evaluate font styles, spacing, element alignment, and visual consistency across modules.
3. Perform a walkthrough with 3 representative users (radiologist, admin, technician).
4. Track time taken to locate a core function (e.g., report viewer).
5. Confirm no modules exhibit layout shifts, overlap, or off-brand UI elements.
6. Success metric: All users navigate without confusion and UI passes visual QA checklist.

4.2.2 Usability and Humanity Requirements

NFRTC3
Title: Ease of Performing Core Functionality Without Guidance
Control: Manual
Initial State: The web application is deployed and accessible via a browser; users are interacting with it for the first time.
Input: Untrained users attempt to upload a chest X-ray and request classification using the tool.
Output: At least 80% of users complete the task within 1 minute without requiring assistance.
Test Case Derivation: Derived from NF-EUR0 - Core functionality should be self-evident and uncluttered for new users.
How the test will be performed:

1. Recruit 5-7 first-time users (non-developers, preferably clinicians or technicians).
2. Instruct them to upload a chest X-ray image and trigger the diagnosis feature without guidance or tutorial.

3. Time the task and record completion success/failure.
4. Observe confusion points or UI obstacles.
5. Validate success if at least 80% complete the task in under 1 minute unaided.
6. Collect optional feedback via Likert-scale survey (e.g., "The task was easy to complete.").

NFRTC4
Title: Intuitiveness and Memorability of the Software
Control: Manual
Initial State: A cohort of users has completed 1-2 guided sessions with the software.
Input: Users attempt to navigate and use the system independently after a one-month break.
Output: Users are able to recall navigation and perform core actions (e.g., upload, view report) with no external aid.
Test Case Derivation: Based on NF-EU2 - The tool must be intuitive enough that users retain task workflows post-training.
How the test will be performed:

1. Provide 1-2 structured walkthroughs of the app to a test group (e.g., radiology trainees).
2. Wait 4 weeks without further exposure.
3. Ask users to independently upload an image and access the report dashboard.
4. Observe interactions and task success.
5. Survey user experience, focusing on recall of navigation flow and interface elements.
6. Pass criteria: At least 80% of participants complete all steps correctly with minimal hesitation.

NFRTC5
Title: Learning Curve Assessment
Control: Manual
Initial State: System is deployed with a functional user interface, and no help prompts are enabled.
Input: First-time users (not involved in design) try to perform a full diagnosis cycle (upload, classify, view result).
Output: Users complete the cycle independently within 1 minute, with high confidence.
Test Case Derivation: Derived from NF-LR0 - The learning curve should be short enough to allow effective tool usage with minimal instruction.
How the test will be performed:

1. Introduce new users to the platform without providing any tutorials or hints.
2. Ask them to diagnose a chest X-ray image.

3. Record time taken and whether they required external help.
4. Collect post-task confidence ratings (e.g., "How confident were you in your actions?" on a 5-point scale).
5. Define success if majority complete within 1 minute and rate confidence ≥ 4.

4.2.3 Usability and Humanity Requirements

NFRTC6
Title: Realism of Generated Chest X-Ray Images
Control: Manual (with expert evaluation)
Initial State: The diffusion model is operational and capable of generating synthetic chest X-ray images with disease signatures.
Input: A set of generated chest X-ray images using the trained model.
Output: Medical professionals (e.g., radiologists) are unable to consistently distinguish between real and generated images, indicating high visual realism.
Test Case Derivation: Based on NF-UPR0 - Generated images should resemble real-world clinical X-rays.
How the test will be performed:

1. Generate a dataset of at least 20 chest X-ray images using the diffusion model.
2. Mix the generated images with an equal number of real chest X-ray images from public datasets (e.g., MIMIC-CXR, CheXpert).
3. Recruit at least 3 medical experts (radiologists or trained physicians) to review the images in a blinded fashion.
4. Ask experts to classify each image as either "Real" or "Generated."
5. Compute the classification accuracy of experts in identifying the image source.
6. If experts perform at or below random guessing threshold $\leq 60\%$ accuracy, the realism of generated images is considered acceptable.
7. Collect qualitative feedback on visual quality, artifacts, or abnormalities for further refinement.

4.2.4 Performance Requirements

NFRTC7
Title: Image Generation Speed Test
Control: Automatic
Initial State: The system is running in its production environment with backend and inference services available.
Input: A request to generate a synthetic chest X-ray image via the frontend interface.
Output: The image is generated and rendered in the interface within 60 seconds of the

request submission.
Test Case Derivation: Based on NF-SLR0 - The system must respond within clinically reasonable time frames for end users.

How the test will be performed:

1. Initiate an image generation request using the web interface.
2. Programmatically log the timestamp when the request is submitted.
3. Record the timestamp when the generated image is fully rendered in the UI.
4. Compute the total latency (end timestamp - start timestamp).
5. Verify that the latency does not exceed 60 seconds.
6. Repeat the process 20 times under varying system loads and average the results.

NFRTC8
Title: System Stability and Uptime Monitoring
Control: Automatic (Continuous)
Initial State: The application is deployed and accessible on a remote server with uptime tracking enabled.
Input: Continuous automated health check pings at fixed intervals (e.g., every 5 minutes).
Output: Uptime exceeds 99% uptime with no more than 1 hour downtime/week in any rolling 7-day window.
Test Case Derivation: Based on NF-RFTR0 - System should be available to clinicians with high reliability.

How the test will be performed:

1. Deploy an automated health check tool (e.g., UptimeRobot, Prometheus, or custom script) to ping the server every 5 minutes.
2. Log all successful responses and any failed pings over a 30-day period.
3. Calculate uptime as a percentage of total ping attempts.
4. Flag any weeks where downtime exceeds 1 hour downtime/week.
5. Validate logs and alerts to ensure appropriate fault recovery mechanisms were triggered.

NFRTC9
Title: Single Image Generation Capacity Enforcement
Control: Manual and Automatic
Initial State: The system is processing an image generation request.
Input: One user submits a valid request; another user (or same user in a new session) submits a second request simultaneously.

Output: The system rejects the second request with a clear message: "Only one generation job allowed at a time."
Test Case Derivation: Based on NF-CR0 - System must serialize image generation to preserve resource integrity.
How the test will be performed:

1. Begin image generation request A from one browser session.
2. Before the process completes, attempt a second generation request (B) from another tab or user.
3. Observe the system behavior and confirm that request B is rejected with an explanatory message.
4. Ensure that request A proceeds unaffected and completes successfully.
5. Repeat with high concurrency (e.g., simulate 5 users) to test queue or blocking mechanisms.

4.2.5 Security Requirements

NFRTC10
Title: Public Accessibility Without Authentication
Control: Manual
Initial State: The web application is deployed on a publicly accessible domain.
Input: Attempt to access the site from multiple devices and IP addresses without entering credentials.
Output: The application loads successfully and allows users to interact with its core features without requiring login or authentication.
Test Case Derivation: Derived from NF-AR0 - The system must be accessible without user registration or login.
How the test will be performed:

1. Open the web application in incognito/private browsing mode from multiple devices (e.g., laptop, tablet, mobile phone).
2. Attempt to load the homepage and initiate core functionalities (e.g., image generation, report viewing).
3. Repeat the above from different network locations (e.g., home, campus, mobile data).
4. Confirm that at no point is the user prompted to sign in, register, or provide personal information.
5. Verify functionality remains consistent across devices and locations.

NFRTC11
Title: Verification of Non-Collection of Personal Data
Control: Manual and Automatic

Initial State: The application is operational and capable of logging user actions and traffic.
Input: Conduct varied user interactions across multiple sessions.
Output: The system performs all operations without collecting, requesting, or storing personally identifiable information (PII).
Test Case Derivation: Derived from NF-PR0 - The app must not collect any personal information.
How the test will be performed:

1. Interact with all available features of the application over several sessions.
2. Use browser developer tools and proxy monitoring software (e.g., Fiddler, Wireshark) to observe all network requests.
3. Look for requests transmitting any PII (e.g., name, email, IP address, device ID).
4. Examine local and session storage for any retained user data.
5. Review backend logs and database content for evidence of personal data storage.
6. Confirm that neither frontend nor backend components request or store PII under any usage flow.

4.2.6 Supportability Requirements

NFRTC12
Title: Availability and Sufficiency of User Documentation
Control: Manual
Initial State: The system is deployed with embedded or linked documentation accessible from the interface.
Input: Navigate to the user documentation and attempt to follow instructions to operate core features.
Output: Users can understand system functionality (e.g., image generation) and successfully perform tasks without additional training.
Test Case Derivation: Derived from NF-SR0 - The tool must be self-sufficient with adequate instructions for experimentation.
How the test will be performed:

1. Access the user documentation through the web app or external documentation link.
2. Read through sections related to system setup, core usage (image generation), and any adjustable parameters (if applicable).
3. Follow the documentation to perform a core task (e.g., generating a chest X-ray image).
4. Measure the number of steps taken and note any ambiguity or missing information.
5. Conduct a brief user study where 3-5 users follow the documentation unaided and report clarity, gaps, or confusion.

6. Success is measured by at least 80% of users completing a task without needing further help.

4.2.7 Cultural Requirements

NFRTC13
Title: Functionality of Browser-Based Language Translation
Control: Manual
Initial State: The application frontend is built in English and available online.
Input: Use a modern browser to translate the UI to multiple other languages.
Output: The web app renders correctly and all core features function identically after translation.
Test Case Derivation: Derived from NF-CR0 - The tool should support global usability without region-locking or linguistic barriers.
How the test will be performed:

1. Open the web app in a browser like Google Chrome or Edge.
2. Enable the browser's built-in translation feature and choose a language (e.g., Spanish, French, Hindi).
3. Observe how the interface text updates across menus, buttons, and messages.
4. Perform several core tasks (e.g., generate image, view heatmap) and ensure no UI breakage or functional errors.
5. Repeat with at least two other languages and browsers.
6. Document any untranslated content or layout inconsistencies that occur.

4.3 Untested Non-Functional Requirements

Some non-functional requirements are acknowledged but excluded from validation due to current project constraints. These constraints include lack of long-term deployment, restricted access to infrastructure, and project scope boundaries. The following are recognized but untested:

- **NF-SER0 (Scalability or Extensibility Requirements):** Testing for horizontal/vertical scaling, modularity, or plugin compatibility is outside current implementation scope.
- **NF-LR0 (Longevity Requirements):** Validating performance and reliability over a three-year timeline is infeasible within this capstone cycle.
- **NF-MR0 (Maintenance Requirements):** Assessing ease of future updates, bug fixes, and dependency upgrades requires an operational phase post-deployment.

- **NF-EPE0 (Expected Physical Environment):** The software is environment-agnostic and runs entirely within a digital infrastructure. No real-world physical testing required.
- **NF-RIAS0 (Interfacing with Adjacent Systems):** No external system integration exists in the current design. Therefore, interfacing behavior falls outside of scope.
- **NF-IR0 (Integrity Requirements):** The system outputs anonymized visual data only. Since no patient identifiers or modifiable inputs are used, specific data integrity testing was deemed unnecessary.
- **NF-AR0 (Audit Requirements):** While security is enforced via encryption and role-based access, no formal third-party audit or audit trail requirement was defined for this phase.

4.4 Traceability Between Test Cases and Requirements

Functional Requirement	**Test Cases**
FR1: The system must allow users to upload chest X-ray images in supported formats (DICOM, JPG, PNG).	FRTC1, FRTC2
FR2: The system must preprocess images by resizing, normalizing, and converting to grayscale if required.	FRTC2
FR3: The system must classify diseases using a CNN model and return predictions with confidence scores.	FRTC3, FRTC4
FR4: The system must support multi-disease classification for a single image.	FRTC5, FRTC6
FR5: The system must display predicted labels, confidence scores, and heatmaps in a readable format.	FRTC10
FR6: The system must generate Grad-CAM heatmaps and overlay them on the original image.	FRTC9
FR7: The system must store image, prediction, heatmap, and report securely for user access.	FRTC13, FRTC14
FR8: The system must authenticate users and implement role-based access control.	FRTC11, FRTC12, FRTC15, FRTC16, FRTC17
FR9: The system must offer a secure API endpoint to receive image inputs and return JSON predictions.	*Not directly tested in current plan*
FR10: The system must gracefully handle errors such as invalid uploads and display informative notifications.	FRTC8
FR11: The system must provide a user dashboard to view past uploads and diagnostic results with timestamps.	FRTC10, FRTC11
FR12: The system must notify users when model confidence is low ($\leq 50\%$) and suggest radiologist review.	FRTC6

Table 2: Traceability Matrix - Functional Requirements to Test Cases

4.5 Traceability Between Test Cases and Requirements

Table 3: Traceability Between Non-Functional Requirements and Test Cases

Non-Functional Requirement	**Test Cases**
NF-AR0: Unauthorized User Access Prevention	FRTC12, FRTC16, NFRTC10
NF-AR1: Calming and Professional Color Scheme	NFRTC1
NF-AR2: Consistent Fonts and Layout	NFRTC2
NF-SR0: Documentation Guide and Self-Sufficiency	NFRTC2, NFRTC12
NF-SR1: Minimalistic and Functional Design	NFRTC2
NF-SR2: Clean, Organized Layout	NFRTC2
NF-EUR0: Efficient Task Completion Without Distractions	NFRTC3
NF-EU2: Intuitive After Minimal Training	NFRTC4
NF-LR0: Learnability and Uptime Expectation	NFRTC5, NFRTC8
NF-UPR0: Realism of Generated Chest X-Rays	NFRTC6
NF-SLR0: 60-second Image Generation Time	NFRTC7
NF-RFTR0: Robustness and Availability	NFRTC8
NF-CR0: Image Generation One-at-a-Time	NFRTC9
NF-CR1: Cultural and Language Flexibility	NFRTC13
NF-PR0: No Personal Data Collected	FRTC13, FRTC14, NFRTC11
NF-IR0: Data Integrity and Access Limits	FRTC13, NFRTC11
NF-AUR0: Audit Logging and Role Restrictions	FRTC16, FRTC17
NF-RIAS0: PACS/EHR Integration	FRTC13
NF-RIAS1: Importing Public Medical Datasets	FRTC1
NF-REL0: Rev 1 Core Feature Validation	FRTC1, FRTC3, FRTC4, FRTC5
NF-REL1: Configurable Logging and API Controls	FRTC13
NF-MR0: Modular System Architecture	FRTC13, FRTC14
NF-MR1: Git-Based Version Control	NFRTC12
NF-SCR0: HL7/DICOM/FHIR Compliance	FRTC4, FRTC5
NF-SER0: Scalability for Future Revisions	Untested
NF-LR1: HIPAA and PIPEDA Compliance	FRTC13, FRTC14

5 Unit Test Description

This section outlines the approach and rationale for the unit tests designed to verify each module within the chest X-ray diagnostic application. The overall philosophy for test case selection centers on ensuring each module functions correctly in isolation, covering both standard functionality and potential edge cases. This approach provides a robust foundation to identify and address any issues at the unit level before integrating modules into the broader system.

To streamline testing and avoid redundancy, each module's tests are designed to validate both typical and boundary conditions. This strategy includes creating a baseline test case for normal expected behavior, accompanied by additional edge cases to capture unexpected or extreme inputs. This allows for comprehensive testing without unnecessary detail for every possible scenario.

For efficiency, test cases prioritize high-risk functions and core functionalities. Tests are organized to:

- Validate standard functionality with a single test for typical cases.
- Test critical edge cases that reflect likely and realistic error conditions (e.g., handling of empty data, invalid formats, and unexpected large data inputs).
- Ensure modules perform within expected limits for nonfunctional requirements, such as performance and security, through specialized tests.
- Include module-specific performance tracking across variable dataset conditions to ensure stability and reliability.
- Evaluate modular interactions such as dataset loading, model persistence, and API response validation.

Where possible, detailed input/output descriptions are omitted in favor of referring to well-documented unit testing code, which maintains readable and meaningful test names. This approach not only saves space but also facilitates future maintenance and expansion, as new edge cases can be added to the codebase directly.

5.1 Unit Testing Scope

Unit testing for this project targets the core components of the chest X-ray diagnostic pipeline, ensuring correctness, resilience, and modularity of each individual module. The scope explicitly addresses all critical stages from data ingestion and transformation to model training and persistence, aligned with the architecture described in the updated SRS and design documents.

Each module is tested in isolation under both standard and edge-case conditions. These unit tests are critical for identifying logic flaws, integration risks, and regressions early in the development lifecycle. By verifying core functionalities before system-wide testing, we ensure

each module behaves predictably and performs as expected across varied dataset conditions, such as missing files, malformed inputs, and irregular class distributions.

Unit tests span the following core components:

- **Data Preprocessing Module** - This module handles the transformation of raw dataset metadata into structured inputs for modeling. Tests focus on:
  - Conversion of disease labels into binary one-hot encoded matrices, ensuring alignment with D disease categories (e.g., `test_create_binary_matrix`).
  - Preservation of patient/image count during training-validation splitting (`test_split_train_val` even in imbalanced or stratified subsets.
  - Proper directory creation and file organization into train/val folders to support reproducible training (`test_organize_images`).
- **ChestXrayDataset Module** - This PyTorch-compatible dataset loader encapsulates image preprocessing, label loading, and transformation logic.
  - `test_dataset_initialization` ensures correct parsing of image paths, label matrices, and class names.
  - `test_dataset_getitem` verifies image size normalization, transformation pipeline output, and label consistency across samples.
- **Model Architecture Module** - Defines the neural architecture used for disease classification, currently based on a ResNet variant.
  - `test_model_structure` validates the number of trainable parameters, output dimensions, and layer names (e.g., whether final FC layer is (D)-dimensional).
  - `test_layer_freezing` confirms transfer learning setup, i.e., initial layers are frozen, later layers are fine-tuned.
  - `test_forward_pass` confirms the output shape from a forward pass and checks for NaN/Inf values in logits.
- **Training and Inference Module** - This module oversees training loop logic and real-time predictions.
  - `test_training_step` runs a minimal 1-epoch training simulation to detect any early training exceptions or convergence issues.
  - `test_prediction` evaluates output softmax/logit distributions on a validation sample, ensuring valid probabilities or class indices are returned.
- **Model Persistence Module** - Verifies model saving and loading logic using torch serialization.
  - `test_save_and_load_model` ensures model weights and architecture are preserved after being saved and reloaded, confirming byte-level equivalence.

- **Data Download and Extraction Module** - Assesses the project's automated data fetching pipeline.
  - `test_data_extraction` checks whether ZIP archives containing image-label data are extracted with proper folder structure.
  - `test_data_download` simulates retrieval from remote servers and verifies final dataset integrity and organization.

Modules that are not included in the unit testing scope are:

- **Third-Party Libraries and External APIs** - Any external libraries or APIs used in the system are assumed to have been verified by their respective developers. These components are not unit-tested here but are included in system and integration testing.

5.2 Tests for Functional Requirements

This section lists representative unit tests grouped by module, highlighting both standard and edge case coverage.

5.2.1 Data Preprocessing Module

1. **Test Name:** Create Binary Label Matrix
   Type: Automatic
   Initial State: Dataset with labeled pathology classes
   Function Tested: `create_binary_matrix`
   Input: List of image samples and associated disease labels
   Expected Output: Binary matrix of shape `(N, D)` where `D` is the number of disease classes
   Test Case Derivation: Validates multilabel one-hot encoding behavior; important for classifier input compatibility.
   How test will be performed: Feed known label sets and verify matrix content and shape.
   Edge Case: Image samples with null/missing disease fields
   Expected Output for Edge Case: Rows with all-zero encoding; handled without exception

2. **Test Name:** Split Dataset for Training and Validation
   Type: Automatic
   Initial State: Complete dataset with labeled examples
   Function Tested: `split_train_val_data`
   Input: Dataset of N samples
   Expected Output: Approximately 80/20 split, preserving label distribution
   Test Case Derivation: Ensures stratified random splitting without class imbalance.
   How test will be performed: Compare label distributions before and after split.
   Edge Case: Datasets with ¡10 samples

Expected Output for Edge Case: Fallback logic maintains at least one validation sample with warning logged.

3. **Test Name:** Organize Image Directory Structure
   Type: Automatic
   Initial State: Image files located in flat directory
   Function Tested: `organize_images`
   Input: Path to unsorted image directory
   Expected Output: Images moved to `train/` and `val/` folders based on metadata
   Test Case Derivation: Confirms file-level reorganization matches label splits.
   How test will be performed: Track moved file paths and match with metadata index.
   Edge Case: Directory write permissions denied
   Expected Output for Edge Case: Logs permission error and halts with safe exit

5.2.2 ChestXrayDataset Module

This module handles dataset creation and data loading. It ensures that chest X-ray images and their corresponding labels are correctly paired and transformed. The tests verify dataset initialization, data retrieval, and format consistency.

1. **Test Name:** Dataset Initialization Verification
   Type: Automatic
   Initial State: Dataset CSV and image files are available and accessible.
   Function Tested: `ChestXrayDataset.__init__`
   Input: A CSV file containing image-label mappings and a valid image directory.
   Expected Output: The dataset initializes successfully with correct sample count and class mappings.
   Test Case Derivation: Validates that all samples are parsed and labels are correctly assigned.
   How the test will be performed:
   (a) Load the dataset with a known subset (e.g., 100 labeled entries).
   (b) Verify the total number of samples and unique classes.
   (c) Confirm no missing image-label pairs exist.

   Edge Case: CSV contains a reference to a missing image file.
   Expected Output for Edge Case: The dataset skips the missing image or raises a descriptive warning.

2. **Test Name:** Sample Retrieval and Transformation
   Type: Automatic
   Initial State: Dataset object is instantiated.

Function Tested: `ChestXrayDataset.__getitem__`
Input: An index to retrieve a sample from the dataset.
Expected Output: Returns a valid image tensor and correct one-hot encoded label vector.
Test Case Derivation: Ensures sample loading, preprocessing (e.g., resizing, normalization), and label pairing are correct.
How the test will be performed:

(a) Retrieve a sample at a specific index.

(b) Verify image tensor shape (e.g., $1 \times 224 \times 224$).

(c) Verify label tensor shape (e.g., `NUM_CLASSES` $\times$ 1) and correct class mapping.

Edge Case: Index out of dataset bounds.
Expected Output for Edge Case: Raises an `IndexError` or returns a controlled exception message.

5.2.3 Model Architecture Module

This module defines the structure and configuration of the disease classification network, `OptimizedChestXRayResNet`. These tests ensure architectural correctness, layer freezing behavior, and forward pass consistency.

1. **Test Name:** Model Architecture Structure Validation
Type: Automatic
Initial State: Model instantiation routine is accessible
Function Tested: `OptimizedChestXRayResNet.__init__`
Input: Symbolic constant `D` for number of disease classes
Expected Output: Model contains input layer, frozen feature extractor layers, and final output layer of size `D`
Test Case Derivation: Verifies structure required for multi-label classification output layer.
How the test will be performed:

(a) Instantiate the model with `D = 4` (for 4 diseases).

(b) Traverse layer modules and assert expected configuration (e.g., ResNet backbone with modified head).

(c) Confirm that the final fully connected layer shape is `(batch\_size, D)`.

Edge Case: Invalid class count (`D = 0` or negative)
Expected Output for Edge Case: Initialization raises `ValueError` with descriptive message

2. **Test Name:** Layer Freezing Enforcement
   Type: Automatic
   Initial State: Model is initialized
   Function Tested: `freeze_feature_extractor_layers`
   Input: None (called during model setup)
   Expected Output: Early convolutional layers are frozen, final classifier layers are trainable
   Test Case Derivation: Ensures transfer learning initialization behaves as expected
   How the test will be performed:
   (a) Iterate through model layers.
   (b) Assert `requires_grad = False` for feature extractor layers.
   (c) Assert `requires_grad = True` for classifier head.

   Edge Case: All layers set to trainable by mistake
   Expected Output for Edge Case: Test fails with assertion error; highlights training inefficiency risk

3. **Test Name:** Forward Pass Shape and Range Validation
   Type: Automatic
   Initial State: Model initialized and in evaluation mode
   Function Tested: `forward`
   Input: Batch of normalized image tensors (e.g., shape `(B, 1, 224, 224)`)
   Expected Output: Tensor of shape `(B, D)` with logits in expected numerical range
   Test Case Derivation: Confirms compatibility between input format and model output
   How the test will be performed:
   (a) Generate dummy input batch with random values.
   (b) Run forward pass.
   (c) Assert output shape equals `(B, D)` and values are finite (no `NaN`).

   Edge Case: Input tensor of incorrect shape
   Expected Output for Edge Case: Raises `RuntimeError` with expected dimension mismatch warning

5.2.4 Training and Inference Module

This module verifies correct model training behavior, prediction validity, and classification integrity. Tests focus on training loop stability, forward pass outputs, and performance under minimal and varied conditions.

1. **Test Name:** Run Single Training Step
   Type: Automatic
   Initial State: Model instantiated, optimizer and dataloader configured
   Function Tested: `training_step`
   Input: Batch of input images and corresponding label vectors
   Expected Output: Loss computed and backward pass executed without error
   Test Case Derivation: Confirms training loop executes correctly on a single batch.
   How the test will be performed:
   (a) Create synthetic dataset batch and pass through `training_step`.
   (b) Assert loss is computed and gradients are non-zero for trainable parameters.

   Edge Case: Batch with all-zero labels
   Expected Output for Edge Case: Loss still computable; model does not crash or overfit to null output

2. **Test Name:** Predict on Validation Set
   Type: Automatic
   Initial State: Trained model in evaluation mode
   Function Tested: `predict`
   Input: Validation batch of normalized X-ray tensors
   Expected Output: Multi-label classification output of shape `(B, D)` with softmax/sigmoid activation values in [0, 1]
   Test Case Derivation: Ensures inference logic matches classification output expectations.
   How the test will be performed:
   (a) Feed a batch through the model's `predict` function.
   (b) Assert the shape and numerical range of the outputs.
   (c) Optionally threshold the probabilities and check for expected class predictions.

   Edge Case: Batch contains noisy or poor-quality X-rays
   Expected Output for Edge Case: Model still produces confidence scores; logs uncertainty without crash

5.2.5 Model Persistence Module

These tests ensure the model's weights, structure, and predictions are retained correctly after saving and reloading. This is critical for reproducibility and real-world deployment of the diagnostic system.

1. **Test Name:** Save and Reload Model File
   Type: Automatic
   Initial State: Trained model with fixed weights and architecture

Function Tested: `torch.save` and `torch.load` or equivalent checkpointing logic
Input: Trained model object and dummy input tensor
Expected Output: Predictions remain consistent before and after model reload
Test Case Derivation: Validates persistence mechanism preserves internal state for deployed models.
How the test will be performed:

(a) Train model minimally or load pretrained checkpoint.

(b) Save weights to disk and reload into a fresh model instance.

(c) Pass identical input tensor before and after reloading.

(d) Compare outputs using a numerical tolerance threshold.

Edge Case: Saved checkpoint partially corrupted or missing layers
Expected Output for Edge Case: Raises descriptive loading error or handles via fallback/exception handling

2. **Test Name:** Check Compatibility of Reloaded Model with Predict Function
Type: Automatic
Initial State: Reloaded model from disk checkpoint
Function Tested: `predict` on reloaded model instance
Input: Validation tensor (same shape as training input)
Expected Output: Valid classification vector of shape `(B, D)` with values in [0, 1]
Test Case Derivation: Ensures reloaded model remains usable in downstream prediction functions.
How the test will be performed:

(a) Reload model from saved state.

(b) Run `predict` on standard validation data.

(c) Confirm output matches expected shape and contains no NaNs/infs.

Edge Case: Reloaded model is mistakenly used in training mode
Expected Output for Edge Case: Logs warning if dropout/batchnorm active; passes if prediction results remain valid

5.2.6 Data Download and Extraction Module

This module handles dataset acquisition and integrity verification. Tests ensure external data sources are correctly fetched, extracted, and structured for downstream use.

1. **Test Name:** Simulate Dataset Download
Type: Automatic
Initial State: System with internet connectivity and temporary storage path configured

Function Tested: `download_dataset`
Input: Simulated or mocked download URL and target directory
Expected Output: Dataset archive successfully downloaded to target path
Test Case Derivation: Ensures remote retrieval logic is functional, even under network throttling.
How the test will be performed:

(a) Use a mocked URL or mirror of a known dataset.

(b) Trigger `download_dataset`.

(c) Confirm file presence at target location and match expected checksum or file size.

Edge Case: URL inaccessible or timeout
Expected Output for Edge Case: Logs download failure and retries or exits with a descriptive message

2. **Test Name:** Extract Dataset Archive
Type: Automatic
Initial State: Downloaded ZIP or TAR archive located on disk
Function Tested: `extract_dataset`
Input: Path to archive and target extraction directory
Expected Output: Files correctly extracted to nested folders with preserved structure
Test Case Derivation: Ensures compatibility with ZIP and TAR formats, verifying extraction structure.
How the test will be performed:

(a) Prepare a test archive with expected folder hierarchy and dummy X-ray images.

(b) Call the extraction utility.

(c) Verify output directory matches structure and file count of the original.

Edge Case: Archive is corrupted or partially downloaded
Expected Output for Edge Case: Raises decompression error; deletes partial output and logs reason for failure

5.3 Tests for Nonfunctional Requirements

This section contains unit-level and behavior-focused tests for nonfunctional requirements, emphasizing performance, interpretability, fairness, and security guarantees. Test coverage includes scenarios defined in the updated SRS and traces directly to previously untested or under-specified areas.

5.3.1 Performance Testing - PredictiveModel Module

1. **Test Name:** Bulk Image Processing Speed
   Type: Automatic
   Initial State: Model loaded with GPU acceleration enabled
   Input: Batch of $B = 32$ chest X-ray images (randomly sampled, resolution 224x224)
   Expected Output: Batch inference completes in < 10 seconds per batch
   How test will be performed: Timer starts before batch is passed to the model. End-to-end latency (inference + post-processing) is recorded.
   Edge Case: Maximum allowable batch size (e.g., GPU memory saturation)
   Expected Output for Edge Case: System gracefully reduces batch size, logs memory error, and retries without crashing

5.3.2 Interpretability Testing - DiagnosticAnalysis Module

1. **Test Name:** Grad-CAM Heatmap Generation
   Type: Automatic (Visual and Structural Validation)
   Initial State: Trained classification model loaded, intermediate activations available
   Input: Chest X-ray with ground truth labels
   Expected Output: Heatmap correctly overlays areas relevant to predictions, highlighting abnormal regions
   How test will be performed: Compare Grad-CAM output for each disease class to expected anatomical focus (e.g., lungs for pneumonia). Quantitatively check localization with Intersection over Union (IoU) if bounding boxes are available.

5.3.3 Fairness and Bias Testing - PredictiveModel Module

1. **Test Name:** Bias Detection Across Demographics
   Type: Automatic (Stratified Evaluation)
   Initial State: Labeled dataset includes demographic metadata (age, sex, ethnicity)
   Input: Validation set split into demographic subgroups
   Expected Output: No subgroup has $> 10\%$ deviation in sensitivity/specificity compared to overall model performance
   How test will be performed: Calculate sensitivity, specificity, precision, and recall for each demographic. Compare metrics and flag significant disparities.
   Edge Case: Subgroup with very small sample size (e.g., < 30 images)
   Expected Output for Edge Case: Logs insufficient data for statistical comparison and skips unreliable metrics

5.3.4 Security Testing - Training and Inference Module

1. **Test Name:** Unauthorized Access Block
   Type: Automatic

Initial State: System initialized without active authentication token
Input: Attempt to access protected endpoints (e.g., download report, view sensitive logs)
Expected Output: 403 Forbidden response with corresponding audit log entry
How test will be performed: Attempt requests using a non-authenticated client or expired token. Verify HTTP status code and ensure access is denied. Check that security logs correctly record the failed access attempt.
Edge Case: Replay attack or malformed token
Expected Output for Edge Case: Denied access with "token invalid or expired" error message; logs reflect the attempt type for forensic traceability

5.4 Traceability Between Test Cases and Modules

Table 4: Legend for Module Labels

Module Label	Description
M1	Data Preprocessing Module - Handles metadata cleaning, label encoding, dataset organization, and train/val splitting.
M2	ChestXrayDataset Module - PyTorch-compatible loader for on-the-fly image loading and transformation.
M3	Model Architecture Module - Defines the classification model architecture (e.g., ResNet-18 variant).
M4	Training and Inference Module - Executes training loop, evaluation, prediction, and bias testing.
M5	Model Persistence Module - Saves and restores trained models using serialization.
M6	Data Download and Extraction Module - Downloads, extracts, and organizes raw datasets.

Table 5: Traceability Matrix for Functional Requirement Test Cases (FRTC)

FRTC	**M1**	**M2**	**M3**	**M4**	**M5**	**M6**
FRTC1	X			X		X
FRTC2	X					
FRTC3		X			X	X
FRTC4		X	X			
FRTC5		X		X		X
FRTC6		X		X		X
FRTC7			X			
FRTC8			X			
FRTC9		X				
FRTC10			X			
FRTC11			X			
FRTC12			X	X		
FRTC13				X		
FRTC14				X		
FRTC15				X		
FRTC16				X		
FRTC17				X		

Table 6: Traceability Matrix for Non-Functional Requirement Test Cases (NFRTC)

NFRTC	**M1**	**M2**	**M3**	**M4**	**M5**	**M6**
NFRTC1	X					
NFRTC2		X			X	
NFRTC3			X			
NFRTC4				X		
NFRTC5	X			X		
NFRTC6		X			X	
NFRTC7				X		
NFRTC8				X		
NFRTC9					X	
NFRTC10			X			
NFRTC11				X		

6 Appendix

In this section, additional information that complements the V&V Plan is included.

6.1 Symbolic Parameters

The definition of test cases will call for certain symbolic constants. Their values are defined in this section for easy maintenance. See the following table for reference.

Symbolic Name	Definition or Description
DATASET-NAME	The chest X-ray imaging datasets used for development and testing. Details are as follows:
MIMIC-CXR	A standard labeled chest X-ray dataset commonly used for medical imaging tasks.
Chest ImaGenome	An extension of MIMIC-CXR, with detailed annotations including 29 bounding boxes on anatomical regions. This dataset links textual report segments to specific bounding boxes, supporting tasks in segmentation and disease localization.

6.2 Usability Survey Questions?

The following survey should be filled out by users after using the system for 5 to 15 minutes. This feedback helps evaluate the user experience and identifies areas for improvement.

6.2.1 User Experience Survey

Time using system: Please provide a rating between 0 and 10 for each of the following categories, with 0 being very difficult or undesirable, and 10 being very easy or desirable.

1. Ease of Use: 0 1 2 3 4 5 6 7 8 9 10
   (0 = very difficult to use, 10 = very easy to use)
2. Navigation: 0 1 2 3 4 5 6 7 8 9 10
   (0 = hard to find what you're looking for, 10 = easy to find what you're looking for)
3. Readability: 0 1 2 3 4 5 6 7 8 9 10
   (0 = hard to understand information, 10 = very easy to understand)
4. Look and Feel: 0 1 2 3 4 5 6 7 8 9 10
   (0 = visually unappealing, 10 = visually appealing)

6.2.2 Additional Feedback:

Notes: (Space for users to leave any additional comments or suggestions about the system.)

Appendix — Reflection

Current Content: Summarizes lessons learned and challenges.
Update Needed: Expand reflections to cover **challenges in classification generalization, optimizing hyperparameters, and improving model transparency for clinical adoption**.

6.3 Question 1

What went well while writing this deliverable?

Harrison Chiu: Planning out the modules for functional testing went pretty smoothly. Reviewing SOLID principles helped a lot and I was able to structure each module with a clear purpose, which made creating the tests feel straightforward. I even tried some initial tests on my own machine to catch edge cases early on, and it gave me confidence that I was covering all the important scenarios. That fuzz testing experiment helped me feel like I was on the right track.

Hamza Issa: Working on identifying tests for functional requirements felt really intuitive. After we reached a consensus regarding the functional requirements and our ability to implement it, it was quite simple to devise tests that were complete and meaningful, as part of creating functional requirements is that the expectation action or output is typically discrete.

Gurnoor Bal: Working on the SRS verification was actually pretty satisfying because I got to make sure everything was crystal clear. I found a few areas in the SRS that needed improvement, and being able to fix those early felt good. Plus, setting up the traceability matrix was helpful since it kept things organized and made it easy to link each requirement with its specific test case. It felt like things were clicking into place.

Jared Paul: I liked working on the design verification part because it let me focus on making sure the system was organized and easy to work with. I emphasized modularity and made sure each module was self-contained, which will make the system easier to maintain down the road. I also created a checklist to make sure each module followed solid design practices, so I felt like I was covering the important aspects without missing anything.

Ahmad Hamadi: Writing the validation plan was rewarding because I got to think about how people would actually use the tool, especially radiologists. I set up task-based testing and feedback sessions, so we'll get a real sense of how practical the tool is. Using actual datasets to check the model's accuracy felt like a solid choice too. It felt like I was creating a plan that would make sure the tool was both accurate and user-friendly.

6.4 Question 2

What pain points did you experience during this deliverable, and how did you resolve them?

Hamza Issa: I found that in order to create a V&V that was really complete meant that I needed to not just need a basic abstract understanding of the theory around Diffusion

models, rather I needed to actually understand the theory, how it is typically implemented today in research and industry, so much so that I needed to find out the type of data structure to expect as output which would be a rather complex matrix. This entire process of learning was really challenging as I was a novice in the subject. But I managed to solve this by purchasing a course on Udemy regarding Theory and Implementation of Diffusion models which helped me really better understand and write a V&V plan that actually made sense in the context of diffusion models.

Harrison Chiu: A bit of a challenge was figuring out just how detailed each test case needed to be. I kept wondering if I was getting too specific or not specific enough. To fix this, I made a quick checklist of essential edge cases based on the project requirements and ran it by the team. Getting their input reassured me that I was covering the right bases without going overboard.

Gurnoor Bal: The main problem was with some of the requirements that were a bit too general or vague. It was hard to nail down exactly what needed verifying. I decided to check in with our project supervisor to get a better handle on what was expected, and I left a few notes in the document to flag areas we might need to clarify later. That way, the process stayed clear without me having to guess on details.

Jared Paul: One thing that wasn't easy was verifying the interfaces for each module. Some of the connections needed more detail to line up with the rest of the design. To sort this out, I went back and added specific inputs and outputs for each module, which made sure everything matched up well with the Module Guide. It took a bit more time, but in the end, it made the design much clearer.

Ahmad Hamadi: The tricky part was making sure the tests weren't just covering easy scenarios. I wanted to be sure they were meaningful. I ran into a few issues with environment consistency, which caused random test failures. To fix this, I standardized the setup and added a few retries for tests that occasionally flaked. Then I went back to the requirements to make sure the tests were focused on the important functions. This way, the tests ended up being both reliable and relevant.

6.5 Question 3

What knowledge and skills will the team collectively need to acquire to successfully complete the verification and validation of your project? Examples of possible knowledge and skills include dynamic testing knowledge, static testing knowledge, specific tool usage, Valgrind etc. You should look to identify at least one item for each team member.

Several different and diverse skills are required in order to create the verification and validation plan for this Chest X-ray Diffusion Model Research project.

Software Specification Design
Our team will need a strong understanding of software specification design to grasp the project context and outline the required functions and modules. By defining each function's purpose, inputs, and edge cases, we can plan the implementation of functions accurately

according to the specifications. This design phase will serve as the foundation for later development stages.

Unit Test Planning
Developing robust unit tests will be crucial. Our team must create comprehensive tests that ensure each function achieves its intended purpose reliably. This will require us to account for all relevant edge cases to avoid misleading results and ensure the accuracy and completeness of our unit tests as a core skill for our V&V plan.

Software Design Principles
To maintain a flexible and maintainable codebase, we'll need to prioritize software design principles, particularly the SOLID principles. For example, we plan to structure modules to follow the Single Responsibility Principle, which will help ensure clarity and cohesion in the design, allowing future engineers to work with the code more effectively.

Diffusion Model Concepts and Implementation
Understanding both the theoretical and practical aspects of diffusion models will be essential for designing an accurate V&V plan. Our team will need knowledge of the inputs, expected outputs, and common data structures used in similar applications to develop thorough and effective tests.

6.6 Question 4

For each of the knowledge areas and skills identified in the previous question, what are at least two approaches to acquiring the knowledge or mastering the skill? Of the identified approaches, which will each team member pursue, and why did they make this choice?

1. **Software Specification Design**
   Revise existing papers on software specification design, for example material on MIS specifications, and work done by David Parnas Reflect on previous Mcmaster Software Engineering course on software design (i.e. 3A04)
2. **Unit Test Planning**
   Revise testing blogs produced by large tech companies such as Meta, Google and Amazon. These blogs devise the strategies these companies adopt in their creation of effective unit tests. Reflect on previous Mcmaster Software Engineering course, 3S03 Software Testing
3. **Software Design Principles**
   Revise SOLID principles via informational blogs such as DigitalOcean, Medium, and FreecodeCamp Implement personal projects that are small but correctly implement each of the design principles.
4. **Diffusion Model Concepts/Implementation**
   Utilize Introduction to Diffusion Models course on Udemy. This goes over theory and implementation in practice today. Read existing papers on experiments with diffusion models by other researchers that have similar applications

Now, each team member selected a particular method to pursue, and stated their justifications below:

Jared Paul: I'm going to revise more formal and professional software specification design practices by looking at existing papers, such as that from David Parnas. This is because I recall being introduced to him in 2nd year, and found the content really straightforward and informative the, so I thought it would be able to get me up to speed quite quickly today.

Hamza Issa: I chose to revise existing engineering blogs from big tech companies like Amazon, as although there are papers and lots of theory out there for devising unit tests. I think that in designing and creating the best unit tests we can look to companies that are highly dependent on them, to find tests that are not just complete but not overly verbose.

Ahmad Hamadi: I plan on revising my SOLID principles using popular blogs like I've seen from FreeCode camp. I think this would be the simplest way to learn all the principles in a manner that is easy to digest and also easy to implement myself for when I want to experiment and practice.

Harrison Chiu: I am going to try and complete the Udemy course on Introduction to Diffusion models. I have some experience looking at research papers for Diffusion models, but they have been really difficult to understand due to the advanced mathematics. I think using a Udemy course would make it simpler to learn and easily transferable to our project

Gurnoor Bal: I am going to investigate existing research papers in the Diffusion Models space that have some similarity to our application. I think this would not only help me learn the required concepts, but also show me how to actually produce research and an application catered for this specific project. If i were to just watch the Udemy course, I'll understand the concepts, but I would still need to review papers to understand how to go about it for this particular project.